



JMPM TECNOLOGIA

TECNOLOGIA & DESENVOLVIMENTO

DOCUMENTAÇÃO TÉCNICA

Arquitetura

Arquitetura interna do Motor Core GraphQL para Protheus

Versão **3.1.0**

JMPM Tecnologia

05/09/2026

Sumário

Arquitetura

Mutations

Documentação Técnica — Arquitetura interna do motor, pipeline de requisição e modelo de mutations.

Arquitetura

Pipeline de requisição:

```
GET /graphql --> GQLSERVICE (entrypoints/service.entrypoint.tlpp)
--> GqlLexer + GqlParser (core/lexer.tlpp, core/parser.tlpp)
--> GqlValidator (core/validator.tlpp)
--> GqlExecutor (core/executor.tlpp)
--> GqlQueryBuilder (core/query-builder.tlpp)
--> GqlDictionaryReader (core/dictionary-reader.tlpp)
--> Resposta JSON
```

O schema nunca é escrito à mão: `GqlSchemaProvider` (`core/schema-provider.tlpp`) monta os tipos GraphQL de forma preguiçosa a partir de SX2 (tabelas) e SX3 (campos), e os campos de relacionamento a partir de SX9. Tabelas/campos são filtrados pelo `GqlAccessControl` (`core/access-control.tlpp`) contra as listas de bloqueio do `config/graphql-config.json` antes de qualquer cache.

Verificações de permissão por usuário estão fora do escopo deste sub-projeto; o sub-projeto Auth vai adicioná-las, provavelmente como outro método no `GqlAccessControl`.

Mutations

`GqlMutationExecutor` (`core/mutation-executor.tlpp`) é um pipeline de escrita paralelo ao executor de leitura, compartilhando o mesmo lexer/parser e o mesmo `GqlDictionaryReader` / `GqlQueryBuilder`. Ele escreve via `TCSqlExec()` — **não** via `TCQuery`, que quebra de forma não-capturável com qualquer coisa que não seja `SELECT` neste ambiente (confirmado empiricamente; veja as Restrições Globais do plano para a investigação completa) — e reselectiona a linha afetada através do `GqlExecutor:resolveTableField()` existente, de modo que a moldagem da resposta (alias, seleções aninhadas) nunca é duplicada.

Uma tabela é gravável apenas se estiver em `allowMutations` (configuração) E ainda passar pela lista de bloqueio do caminho de leitura — as duas condições se combinam, nenhuma sozinha é suficiente. `GqlInputValidator` verifica obrigatoriedade/tipo/tamanho contra os metadados do SX3 antes de qualquer SQL rodar. Exclusão é sempre lógica (`D_E_L_E_T_ = '*'`), coerente com como toda consulta já filtra leituras.

Campos de entrada desconhecidos ou bloqueados são atualmente ignorados silenciosamente, não rejeitados com erro explícito: `GqlValidator` nunca é invocado no caminho de mutation, então um nome de campo que não faz parte das listas SET/INSERT guiadas pelo dicionário é simplesmente excluído quando essas listas são montadas — nunca chega ao SQL, então isso não é um furo de segurança, mas diverge da semântica normal do GraphQL (um campo desconhecido em um objeto de entrada deveria ser um erro de validação). Fechar isso fica para a validação de argumentos de mutation por campo de um sub-projeto futuro.

Limitação conhecida: Concorrência e atribuição de `R_E_C_N_O_`

Mutations `create` atribuem `R_E_C_N_O_` (a chave primária física da tabela neste caminho de escrita com SQL cru, já que a atribuição automática normal do ISAM/DBAccess é contornada) via uma subconsulta SQL `MAX(R_E_C_N_O_)+1`. Isso **não é seguro sob escrita concorrente** — duas chamadas `create` simultâneas contra a mesma tabela podem entrar em corrida e tentar inserir o mesmo valor de `R_E_C_N_O_`, causando colisão de chave primária.

Tanto uma sequência/IDENTITY real do Postgres quanto uma abordagem de advisory lock na mesma instrução foram investigadas e consideradas indisponíveis/ineficazes neste ambiente. Fechar essa lacuna exige uma migração de banco (adicionar sequências/IDENTITY reais às tabelas de dicionário do Protheus) ou uma stored procedure no servidor — ambos fora do escopo deste sub-projeto.

Garantia de segurança atual: seguro para uso de requisição única e baixa concorrência (ex.: os próprios testes deste sub-projeto). **Não seguro** para tráfego de produção concorrente na mesma tabela.

Veja o cabeçalho Protheus.doc de `buildInsert()` em `custom/backoffice/graphql/core/mutation-builder.tlpp` para a análise técnica completa.